Day 9 — AI Agent Tools and Function Calling

From AI Reasoning to Real Software Actions

Course

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture Duration

60 Minutes

Level

Beginner → Early Intermediate

Day 9 Main Goal

Day 8 introduced the concept of an AI Agent.

Students learned this basic architecture:

```
User Goal
   ↓
AI Agent
   ↓
Observe
   ↓
Decide
   ↓
Choose Tool
   ↓
Act
   ↓
Observe Result
   ↓
Continue / Finish
```

Day 9 focuses on one of the most important parts of that architecture:

TOOLS

Without tools, an LLM mainly works with information available in its context.

With tools, an agent can interact with software systems.

Examples:

```
get_customer()
get_order()
get_tracking()
search_policy()
check_inventory()
create_ticket()
send_email()
```

Current OpenAI function calling allows models to request application-defined functions using structured arguments; the application executes those functions, returns their results to the model, and the model can continue with further calls or produce a final response.

Day 9 Learning Objectives

By the end of the lecture, students should understand:

- What an AI tool is
- What function calling means
- Function vs tool terminology
- Tool definitions
- Tool names
- Tool descriptions
- Tool parameters
- JSON Schema for tool inputs
- Required parameters
- Optional parameters
- Enums
- Strict schemas
- Tool outputs
- Structured tool results
- Tool errors
- Read tools
- Write tools
- Side effects
- Tool permissions

- Human approval
- Tool selection
- Tool ambiguity
- Tool chaining
- Parallel vs sequential calls conceptually
- Retry strategies
- Timeouts
- Idempotency
- Error codes
- Observability
- Tool logs
- API-backed tools
- RAG tools
- Database tools
- n8n workflow tools
- Safe tool design
- Testing agent tools
- Designing a complete support-agent toolset

---

1. Day 9 — 60-Minute Lecture Schedule

Time	Topic
0–5 min	Day 8 review
5–12 min	What is a tool?
12–20 min	Function-calling flow
20–28 min	Designing tool names, descriptions, and parameters
28–35 min	Read vs write tools and permissions
35–42 min	Tool outputs, errors, retries, and validation
42–48 min	Tool selection and multi-tool workflows
48–53 min	APIs, RAG, databases, and n8n as tools
53–57 min	Mini project: Support Agent toolset
57–60 min	Quiz, recap, homework

---

2. Day 8 Review — 0 to 5 Minutes

Ask students:

Question 1

What is an AI Agent?

Expected:

> An AI-powered system that can pursue a goal, decide what it needs, and use available tools.

Question 2

What is an agent loop?

```
Observe
 ↓
Decide
 ↓
Act
 ↓
Observe
 ↓
Continue / Finish
```

Question 3

What is a tool?

Expected:

> A capability that allows the agent to retrieve information or perform an action.

Question 4

What is the difference between a read tool and a write tool?

Read

Retrieves information.

```
get_order()
```

Write

Changes something.

```
cancel_order()
```

Question 5

Why are write tools usually more risky?

Because they can change real-world state.

Examples:

```
Send Email
Cancel Order
Update CRM
Issue Refund
Delete Record
```

---

3. Transition to Day 9

Tell students:

Yesterday we said:

```
Agent
 ↓
Choose Tool
 ↓
Use Tool
```

Today we answer:

> What exactly is a tool?

> How does the model choose it?
>
> How are tool inputs defined?
>
> Who actually executes the tool?
>
> What happens when the tool fails?

---

4. What Is an AI Tool? — 5 to 12 Minutes

Use this simple definition:

> **A tool is a capability exposed to an AI model so it can request information or an action that the model cannot or should not perform by itself.**

Examples:

```
get_weather()
search_documents()
get_customer()
check_inventory()
send_email()
create_invoice()
```

OpenAI's current tooling documentation describes tools as functionality made available to a model; function tools are defined with structured schemas, while other tool types can provide hosted capabilities such as web search, file search, code execution, and MCP integrations.

---

5. Model Without Tools

User:

> Where is order #ABC123?

LLM:

```
I do not have access to the company's current order system.
```

Correct.

---

6. Model With an Order Tool

Available tool:

```
get_order(order_id)
```

Now:

```
Customer
 ↓
Agent
 ↓
get_order()
 ↓
Order System
 ↓
Current Information
 ↓
Agent
 ↓
Answer
```

---

7. Important Distinction

Do not tell beginners:

> GPT calls the database directly.

Instead:

```
MODEL
 ↓
Requests Tool Call
 ↓
APPLICATION
 ↓
Executes Code
 ↓
DATABASE/API
 ↓
```

```
Result
 ↓
MODEL
```

The application controls the real integration.

That separation is fundamental to current function-calling systems.

---

8. Function vs Tool

For beginner purposes:

Tool

General capability available to the AI.

Examples:

```
Web Search
File Search
Database Lookup
Order Lookup
Send Email
```

Function Tool

A specific application-defined function with structured parameters.

Example:

```
get_order(order_id)
```

OpenAI's current API treats function calling as one type of tool interface, with functions defined by JSON Schema.

---

9. Basic Tool Anatomy

Every useful function tool should conceptually have:

```
NAME
+
DESCRIPTION
+
INPUT PARAMETERS
+
EXECUTION LOGIC
+
OUTPUT
```

Optional but important:

```
ERROR BEHAVIOR
PERMISSIONS
APPROVAL RULES
TIMEOUTS
LOGGING
```

---

10. Example Tool

```
Name:
get_order

Description:
Retrieve the latest information for a customer order.

Input:
order_id

Output:
order details
```

---

11. Why Tool Names Matter

Bad:

```
do_thing()
```

The model cannot understand its purpose clearly.

Better:

```
get_order()
```

Even better when tools could overlap:

```
get_order_by_id()
search_customer_orders()
```

---

12. Tool Name Guidelines

Prefer names that are:

- Clear
- Specific
- Action-oriented
- Short
- Unambiguous

Good:

```
get_customer
get_order
get_tracking
search_policy
check_inventory
create_support_ticket
```

Weak:

```
data
process
tool1
handle
do_action
```

---

13. Function Calling Flow — 12 to 20 Minutes

OpenAI documents tool calling as a multi-step interaction:

```
1. Application sends prompt + available tools
2. Model chooses a tool
3. Application executes that tool
4. Application returns tool output
5. Model produces a response or requests another tool
```

The cycle can continue through multiple tool calls when required.

---

14. Full Conceptual Flow

```
User:
"Where is order A829?"
        ↓
Application sends:
User message
+
Available tools
        ↓
Model
        ↓
Tool Call:
get_order
{
  "order_id": "A829"
}
        ↓
Application Executes
        ↓
Order API
        ↓
Result:
{
  "status": "shipped"
}
        ↓
Result returned to model
        ↓
```

```
Model:
"Your order has shipped."
```

---

15. What the Model Produces

The model does not need to generate:

```
database.query(...)
```

Instead it may produce structured tool arguments such as:

```
{
  "order_id": "A829"
}
```

Your application determines what that means operationally.

---

16. Why This Architecture Is Valuable

The application can enforce:

```
Authentication
Permissions
Validation
Rate Limits
Business Rules
Audit Logs
Human Approval
```

before performing the requested action.

---

17. Example — Weather Tool

Tool definition concept:

```
get_weather(location)
```

User:

> What's the weather in Miami?

Agent determines:

```
{
  "location": "Miami"
}
```

Tool executes.

Result:

```
{
  "temperature": 31,
  "unit": "C"
}
```

Agent can produce a natural-language answer.

---

18. Business Example — Order Lookup

User:

> Check order #ORD-99821.

Tool:

```
get_order
```

Arguments:

```
{
  "order_id": "ORD-99821"
}
```

Result:

```
{
  "id": "ORD-99821",
  "status": "processing"
}
```

---

19. The Model Requests — Application Executes

Write this prominently:

> **The model requests the action. The application controls execution.**

This is a central agent-engineering principle.

---

20. Tool Descriptions — 20 to 28 Minutes

Tool descriptions help the model decide when a tool should be used.

Compare:

Bad Description

```
Gets data.
```

Better Description

```
Retrieve the current status and details for a known order ID.
Use this when the user provides a specific order identifier.
```

---

21. Why Description Matters

Suppose we have:

```
get_order
```

and:

```
search_customer_orders
```

Descriptions should clarify the difference.

---

22. `get_order`

```
Retrieve one order when the order ID is already known.
```

---

23. `search_customer_orders`

```
Search a customer's recent orders when the customer is known but no order ID is
available.
```

Now the model has meaningful guidance.

---

24. Avoid Overlapping Tools

Poor toolset:

```
lookup_order
find_order
search_order
get_order
retrieve_order
```

All appear similar.

This increases ambiguity.

Better:

```
get_order_by_id
search_orders_by_customer
```

Each has a distinct responsibility.

---

25. Tool Design Principle

> **One tool should have one clear job.**

Avoid:

```
manage_customer_everything()
```

which can:

- Search customer
- Edit account
- Delete account
- Refund payment
- Send email

This is too broad.

---

26. Better Tool Separation

```
get_customer()
update_customer_email()
create_support_ticket()
issue_refund()
```

Benefits:

- Easier permissions
- Easier testing
- Easier approval rules
- Easier logging
- Clearer agent decisions

---

27. Tool Parameters

A tool must know what information it needs.

Example:

```
get_order(order_id)
```

Input:

```
{
  "order_id": "A123"
}
```

---

28. JSON Schema for Tool Inputs

Current function-calling interfaces use JSON Schema-style parameter definitions so the model can supply structured arguments.

Conceptual schema:

```
{
  "type": "object",
  "properties": {
    "order_id": {
      "type": "string"
    }
  },
  "required": [
    "order_id"
  ]
}
```

---

29. Why Schema Matters

Without structure:

```
Check John's order, maybe the latest one.
```

With structure:

```
{
  "order_id": "ORD-8872"
}
```

Software can validate the second form reliably.

---

30. Required Parameters

Suppose:

```
get_order
```

cannot work without an order ID.

Then:

```
order_id
```

should be required.

If the model does not have it, the correct behavior may be:

```
Ask user
```

or:

```
Use another search tool
```

---

31. Optional Parameters

Tool:

```
search_products()
```

Could have:

```
{
  "query": "running shoes",
  "color": "blue",
  "size": null
}
```

Maybe only:

```
query
```

is required.

---

32. Enums in Tool Parameters

Example:

```
{
  "priority": {
    "type": "string",
    "enum": [
      "low",
      "medium",
      "high"
    ]
  }
}
```

This prevents unexpected values such as:

```
super_urgent
```

---

33. Boolean Parameter

Example:

```
{
  "include_cancelled": false
}
```

Useful for search behavior.

---

34. Numeric Parameter

Example:

```
{
  "refund_amount": 250
}
```

Do not send:

```
{
  "refund_amount": "two hundred fifty dollars"
}
```

when software expects a number.

---

35. Strict Tool Schemas

Modern OpenAI function definitions can use strict schema behavior so generated arguments adhere more closely to the declared structure.

For beginner students, the key principle is:

```
Tool Input
↓
Predictable Schema
↓
Validation
```

```
↓
Execution
```

---

36. Read vs Write Tools — 28 to 35 Minutes

This distinction becomes critical once agents interact with real systems.

---

37. Read Tool

A read tool retrieves information.

Examples:

```
get_customer
get_order
get_tracking
check_inventory
search_policy
```

Usually:

```
External State
Does Not Change
```

---

38. Write Tool

A write tool changes information or causes an external action.

Examples:

```
send_email
cancel_order
create_ticket
update_customer
issue_refund
```

---

39. Tool Side Effect

A side effect means the tool changes something outside the agent's internal computation.

Example:

```
send_email()
```

causes a real email to be sent.

---

40. No Side Effect

```
get_order()
```

only reads information.

---

41. Side-Effect Classification

Teach students to label tools:

```
READ ONLY

LOW-RISK WRITE

HIGH-RISK WRITE
```

---

42. Example Classification

Tool	Type	Risk
get_order	Read	Low
search_policy	Read	Low
create_internal_note	Write	Low
send_email	Write	Medium

Tool	Type	Risk
cancel_order	Write	High
issue_refund	Write	High

---

43. Approval Rules

Potential policy:

```
READ TOOLS
→ automatic

CREATE INTERNAL TICKET
→ automatic

SEND CUSTOMER EMAIL
→ review for selected cases

CANCEL ORDER
→ approval required

ISSUE REFUND
→ approval required
```

The current OpenAI Agents SDK supports human-in-the-loop flows where tool execution can pause until a person approves or rejects a sensitive call.

---

44. Example Approval Flow

```
Agent:
issue_refund(
  amount=1200
)

        ↓

Approval Rule
        ↓

Amount > $500
```

```
        ↓

PAUSE
        ↓

Manager Review
    ┌─────┴─────┐
Approve    Reject
   ↓          ↓
Execute     Stop
```

---

45. Important Security Principle

Do not rely only on:

```
"Please don't refund too much."
```

Use:

```
Application Permissions
+
Workflow Rules
+
Approval Limits
```

---

46. Least Privilege

Give an agent only the tools it needs.

Support agent:

```
get_customer
get_order
search_policy
create_ticket
```

Probably does not need:

```
delete_database
modify_payroll
change_admin_permissions
```

---

47. Tool Permissions Per Agent

Different agents may receive different tools.

Product Agent

```
search_products
check_inventory
search_product_docs
```

Billing Agent

```
get_invoice
get_payment
create_billing_ticket
```

Finance Manager Agent

May have higher-level financial capabilities with stronger approvals.

---

48. Tool Output Design — 35 to 42 Minutes

Tools should return predictable information.

Bad:

```
Looks okay.
```

Better:

```
{
  "success": true,
```

```
  "order": {
    "id": "A123",
    "status": "shipped"
  }
}
```

---

49. Successful Tool Result

Use a predictable pattern:

```
{
  "success": true,
  "data": {
    "order_id": "A123",
    "status": "shipped"
  },
  "error": null
}
```

---

50. Failed Tool Result

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "No matching order was found."
  }
}
```

---

51. Why Error Codes Help

The agent can reason about:

```
ORDER_NOT_FOUND
```

better than an unpredictable sentence.

Workflow:

```
ORDER_NOT_FOUND
      ↓
Ask for another identifier
```

---

52. Common Error Codes

Example support system:

```
ORDER_NOT_FOUND

CUSTOMER_NOT_FOUND

UNAUTHORIZED

INVALID_ARGUMENT

RATE_LIMITED

SERVICE_UNAVAILABLE

TIMEOUT

INTERNAL_ERROR
```

---

53. Do Not Hide Tool Failures

Bad:

```
get_order failed
```

Agent then invents:

> Your order is probably shipping.

Wrong.

Correct:

```
Tool Failure
↓
Do Not Guess
↓
Retry / Ask / Escalate
```

---

54. Timeout

A tool may take too long.

Example:

```
Shipping API
↓
No response
```

Result:

```
{
  "success": false,
  "error": {
    "code": "TIMEOUT"
  }
}
```

---

55. Retry Concept

Some failures are temporary.

Example:

```
SERVICE_UNAVAILABLE
```

The application may retry.

Concept:

```
Attempt 1
 ↓
Fail
 ↓
Wait
 ↓
Attempt 2
```

But retry behavior should be controlled by application logic.

---

56. Do Not Retry Forever

Bad:

```
Retry
Retry
Retry
Retry
Retry
...
```

Use limits.

Example:

```
Maximum 3 attempts
```

then:

```
Escalate
```

---

57. Idempotency — Beginner Introduction

This is an important concept for write tools.

Suppose:

```
issue_refund()
```

is accidentally called twice.

Without protection:

```
Refund $100
+
Refund $100
```

could happen.

That is dangerous.

---

58. What Is Idempotency?

Simple beginner definition:

> Designing an action so repeating the same request does not accidentally perform the business action multiple times.

Example:

```
refund_request_id = "REF-123"
```

If the exact request is repeated, the payment system can recognize:

> Already processed.

This concept is especially important for payments, orders, messages, and other side-effecting operations.

---

59. Read Tool Retry vs Write Tool Retry

Read:

```
get_order()
```

Retrying is often relatively safe.

Write:

```
issue_refund()
```

Retrying requires much more care.

---

60. Validate Tool Results

Do not trust external data blindly either.

Tool returns:

```
{
  "price": "banana"
}
```

Your application may reject it.

Architecture:

```
Tool Result
 ↓
Validate
 ↓
Agent
```

---

61. Tool Selection — 42 to 48 Minutes

An agent may have many tools.

Example:

```
get_customer
get_order
get_tracking
search_policy
check_inventory
create_ticket
```

The challenge:

> Which tool should it choose?

---

62. Example 1

User:

> Where is order #A123?

Best first tool:

```
get_order
```

---

63. Example 2

User:

> Do you have this jacket in XL?

Best tool:

```
check_inventory
```

---

64. Example 3

User:

> Can I return an opened product?

Likely:

```
search_policy
```

---

65. Example 4

User:

> My package arrived damaged and I want help.

Possible tool sequence:

```
get_order
 ↓
search_return_policy
 ↓
check_replacement_inventory
 ↓
create_ticket
```

depending on results.

---

66. Sequential Tool Calling

A tool result determines the next tool.

Example:

```
get_order()
 ↓
tracking_id
 ↓
get_tracking()
```

The second call depends on the first.

---

67. Parallel Tool Calling — Concept

Sometimes two independent pieces of information may be collected together.

Example:

```
get_customer()
+
search_policy()
```

If neither depends on the other's output, systems may be able to retrieve them concurrently.

Students only need the concept today.

---

68. Tool Chain Example

Customer:

> Can I exchange order #X281 for a black XL jacket?

Agent may need:

```
1. get_order(X281)
       ↓
2. search_exchange_policy()
       ↓
3. check_inventory(
     product="jacket",
     color="black",
     size="XL"
   )
       ↓
4. Decide options
```

---

69. Do Not Call Tools Unnecessarily

Customer:

> Thank you for your help.

Wrong:

```
get_customer
get_order
get_tracking
search_policy
```

Correct:

```
No tool required.
```

---

70. Tool Cost Awareness

Every tool call can add:

- Latency
- API cost
- Infrastructure cost
- Failure probability
- Complexity

Therefore:

> Use tools when they add information or necessary action.

---

71. Tool Selection Instructions

Agent instructions can help.

Example:

```
Use get_order only when a specific order ID is known.

Use search_customer_orders when customer identity is known but order ID is
missing.

Use search_policy only for policy questions.
```

```
Do not call tools when the answer is already available.
```

---

72. Agent Needs to Know When NOT to Call a Tool

A professional agent decision may be:

```
No tool required
```

This is just as important as good tool selection.

---

73. APIs as Tools — 48 to 53 Minutes

Most business agent tools eventually connect to APIs.

Example:

```
get_order()
      ↓
Shopify / ERP API
```

---

74. API Tool Architecture

```
Agent
 ↓
Function Tool
 ↓
Python / Node.js
 ↓
HTTP API
 ↓
Business System
 ↓
JSON
 ↓
Tool Result
```

```
  ↓
Agent
```

---

75. Example API Tool Concept

```python
def get_order(order_id):
    response = order_api.get(order_id)

    return {
        "success": True,
        "data": response
    }
```

The exact implementation depends on the external system.

---

76. Database as a Tool

Tool:

```
get_customer(customer_id)
```

Behind the tool:

```
Database Query
```

Agent should not receive unrestricted database access if a narrower function will work.

Better:

```
get_customer(customer_id)
```

than:

```
run_any_sql(sql)
```

for most beginner business agents.

---

77. Why Narrow Database Tools Are Safer

`run_any_sql()` could potentially:

- Read unrelated information
- Modify data
- Delete data
- Expose sensitive records

Narrow tools provide stronger boundaries.

---

78. RAG as a Tool

Tool:

```
search_company_knowledge(query)
```

Architecture:

```
Agent
 ↓
RAG Tool
 ↓
Vector Search
 ↓
Relevant Documents
 ↓
Agent
```

This will be studied deeply during the RAG section.

---

79. RAG Tool Example

Customer:

> Can I return a product after 20 days?

Agent chooses:

```
search_return_policy(
  query="return eligibility after 20 days"
)
```

Tool may return:

```
{
  "source": "returns_policy.pdf",
  "content": "Products may be returned within 30 days..."
}
```

Then the model produces a grounded answer.

---

80. n8n as a Tool

n8n agents can connect tool nodes, and n8n also provides a workflow tool that allows an agent to invoke another n8n workflow and receive its result.

Conceptually:

```
AI Agent
   ↓
Call n8n Workflow Tool
   ↓
Sub-workflow
   ↓
CRM
Email
Database
API
   ↓
Result
   ↓
Agent
```

---

81. Why n8n Workflow Tools Are Useful

Instead of the agent directly understanding a complicated integration:

```
CRM API
+
Authentication
+
Data Transformation
+
Error Handling
```

you can wrap that complexity inside:

```
n8n Workflow
```

and expose a simpler capability:

```
create_crm_lead()
```

---

82. Example

Agent tool:

```
qualify_and_create_lead()
```

Behind it:

```
n8n Workflow
├── Validate Input
├── Check CRM
├── Create Lead
├── Assign Owner
└── Return Result
```

---

83. Tool Abstraction

This illustrates a powerful software concept.

The agent sees:

```
create_lead()
```

It does not need to understand:

```
OAuth
HTTP Headers
CRM Endpoints
Data Mapping
Retry Logic
```

The tool hides that complexity.

---

84. Tool Design Principle

> **Expose business capabilities to the agent, not infrastructure complexity.**

Better:

```
get_customer
```

than:

```
execute_http_get_to_crm_endpoint_7
```

---

85. Current OpenAI Agents SDK Tool Types

At a higher level, the current Agents SDK supports several kinds of tools, including hosted OpenAI tools, locally executed/function tools, and agents exposed as tools.

For beginners, focus first on:

```
Function Tools
```

because they clearly demonstrate:

```
Model
→ structured arguments
→ your code
→ result
```

---

86. Beginner Python Tool Concept

Example:

```python
def get_order(order_id: str):
    """Get an order by its ID."""

    fake_orders = {
        "A123": {
            "status": "shipped",
            "tracking_id": "T900"
        }
    }

    order = fake_orders.get(order_id)

    if not order:
        return {
            "success": False,
            "error": {
                "code": "ORDER_NOT_FOUND"
            }
        }

    return {
        "success": True,
        "data": order
    }
```

This is not yet an agent.

It is simply the business function the agent could use.

---

87. Tool Wrapper Concept

With an agent framework, a normal Python function can be exposed to the model as a callable tool. The current OpenAI Agents SDK supports wrapping Python functions as function tools.

Conceptually:

```
Python Function
      ↓
Tool Definition
      ↓
Agent
```

---

88. Beginner Tool Example

```python
from agents import Agent, Runner, function_tool

@function_tool
def get_order(order_id: str) -> str:
    """Retrieve order information for a known order ID."""
    return f"Order {order_id} is currently processing."

agent = Agent(
    name="Order Support Agent",
    instructions="""
    Help customers with order questions.
    Use get_order when a known order ID is provided.
    Do not invent order information.
    """,
    tools=[get_order],
)
```

This pattern follows the current Agents SDK approach of exposing Python functions as tools.

---

89. Important Teaching Point

Students should understand the architecture even if they cannot yet write the code from memory.

```
@function_tool
      ↓
Turns Function Into Tool
      ↓
Agent Receives Tool
      ↓
Model Can Request It
```

---

90. Tool Error Handling

Imagine:

```
get_order("BAD-ID")
```

Result:

```
{
  "success": false,
  "error": {
    "code": "ORDER_NOT_FOUND"
  }
}
```

Agent should respond appropriately.

---

91. Good Agent Response

> I couldn't find an order with that number. Please verify the order ID or provide the email associated with your purchase.

---

92. Bad Agent Response

> Your package probably shipped yesterday.

Never turn a tool failure into invented data.

---

93. Tool Testing

Each tool should be tested separately before giving it to an agent.

Test:

```
Valid Input
Invalid Input
Missing Input
Unauthorized Input
API Failure
Timeout
Unexpected API Data
Large Input
Duplicate Request
```

---

94. Example Tool Test Table

Test	Input	Expected
Valid order	A123	Success
Unknown order	Z999	ORDER_NOT_FOUND
Empty ID	""	INVALID_ARGUMENT
API down	A123	SERVICE_UNAVAILABLE

---

95. Test the Tool Before Testing the Agent

Wrong approach:

```
Agent fails
↓
Change prompt
```

But the real problem might be:

```
Tool itself is broken
```

Correct process:

```
Test Tool
 ↓
Test Tool Schema
 ↓
Test Agent Tool Selection
 ↓
Test Full Workflow
```

---

96. Tool Observability

For every tool call, useful logs include:

```
Timestamp
Agent
Tool Name
Arguments
Result
Duration
Success / Failure
Approval Status
```

Agent frameworks expose lifecycle hooks and tracing mechanisms around model calls and tool calls to help inspect execution.

---

97. Example Log

```
14:02:01
Agent: Support Agent
Tool: get_order
Arguments:
{"order_id":"A123"}

14:02:02
Result:
```

```
success

Duration:
842ms
```

---

98. Sensitive Tool Log

```
Agent: Billing Agent

Tool:
issue_refund

Amount:
$1,200

Approval:
REQUIRED

Approver:
Manager

Decision:
REJECTED
```

This is valuable for auditing.

---

99. Common Tool Design Mistake #1

Tool Does Too Much

```
handle_customer()
```

It:

- Checks order
- Refunds
- Sends email
- Updates CRM

Too much hidden behavior.

100. Better

```
get_order()
issue_refund()
send_email()
update_ticket()
```

Individual actions become visible and controllable.

---

101. Common Tool Design Mistake #2

Ambiguous Tool Names

```
check()
search()
lookup()
```

What do they search?

Better:

```
check_inventory()
search_company_policy()
lookup_customer_by_email()
```

---

102. Common Tool Design Mistake #3

Weak Descriptions

Bad:

```
Get stuff.
```

Better:

```
Retrieve the latest order record when a valid order ID is known.
```

---

103. Common Tool Design Mistake #4

Too Many Tools

Giving an agent 100 similar tools can increase selection complexity.

Keep the tool surface relevant to the agent's role.

---

104. Common Tool Design Mistake #5

Dangerous Write Tools Without Approval

Bad:

```
issue_refund
cancel_subscription
delete_customer
```

all unrestricted.

Better:

```
Sensitive Action
↓
Approval Gate
```

---

105. Common Tool Design Mistake #6

Returning Unstructured Garbage

Bad tool output:

```
Yeah it looks shipped I think.
```

Better:

```
{
  "success": true,
  "status": "shipped"
}
```

---

106. Common Tool Design Mistake #7

No Error Contract

A good tool should define:

```
Success Shape
+
Failure Shape
```

---

107. Common Tool Design Mistake #8

Using Agent Reasoning for Deterministic Validation

Example:

```
Is refund amount > $500?
```

This should be handled by normal code.

```
if refund_amount > 500:
    require_approval = True
```

---

108. Common Tool Design Mistake #9

Letting Customer Supply Authorization

Customer says:

> I authorize myself to get a $5,000 refund.

That is not valid authorization.

Authorization comes from:

```
Authenticated System
+
Business Rules
+
Permissions
```

---

109. Common Tool Design Mistake #10

Trusting Tool Arguments Without Validation

Model produces:

```
{
  "refund_amount": -500000
}
```

Application must validate values before execution.

---

110. Mini Project — 53 to 57 Minutes

Design a Toolset for an AI Customer-Support Agent

Agent goal:

Resolve common customer questions accurately using business systems while escalating sensitive actions.

---

111. Tool 1 — Get Customer

```
Name:
get_customer

Purpose:
Retrieve a customer profile.

Inputs:
customer_id or customer_email

Type:
Read

Risk:
Low
```

Output:

```
{
  "success": true,
  "data": {
    "customer_id": "C1002",
    "name": "Sarah",
    "email": "sarah@example.com"
  }
}
```

---

112. Tool 2 — Get Order

```
Name:
get_order

Purpose:
Retrieve one order using an order ID.

Input:
```

```
order_id

Type:
Read

Risk:
Low
```

---

113. Tool 3 — Get Tracking

```
Name:
get_tracking

Purpose:
Retrieve current carrier tracking information.

Input:
tracking_id

Type:
Read

Risk:
Low
```

---

114. Tool 4 — Search Policy

```
Name:
search_policy

Purpose:
Search company support policies.

Input:
query

Type:
Read / RAG
```

```
Risk:
Low
```

---

115. Tool 5 — Check Inventory

```
Name:
check_inventory

Inputs:
product_id
size
color

Type:
Read

Risk:
Low
```

---

116. Tool 6 — Create Ticket

```
Name:
create_support_ticket

Inputs:
customer_id
category
priority
summary

Type:
Write

Risk:
Low/Medium
```

---

117. Tool 7 — Send Email

```
Name:
send_customer_email

Inputs:
customer_id
subject
body

Type:
Write

Risk:
Medium
```

Potential approval:

```
High-risk case?
→ Review before sending
```

---

118. Tool 8 — Cancel Order

```
Name:
cancel_order

Input:
order_id

Type:
Write

Risk:
High

Approval:
Required
```

---

119. Tool 9 — Issue Refund

```
Name:
issue_refund

Inputs:
order_id
amount
reason

Type:
Write

Risk:
High

Approval:
Required
```

---

120. Complete Tool Table

Tool	Type	Risk	Approval
get_customer	Read	Low	No
get_order	Read	Low	No
get_tracking	Read	Low	No
search_policy	Read	Low	No
check_inventory	Read	Low	No
create_support_ticket	Write	Low/Med	Maybe
send_customer_email	Write	Medium	Conditional
cancel_order	Write	High	Yes
issue_refund	Write	High	Yes

---

121. Mini Project Scenario

Customer says:

> Hi, order #A123 was supposed to arrive three days ago. Tracking hasn't updated. If it won't arrive tomorrow, I want to cancel it.

Ask students:

> What tools should the agent use?

---

122. Expected Path

```
1. get_order("A123")
       ↓
2. Retrieve tracking_id
       ↓
3. get_tracking(tracking_id)
       ↓
4. Determine whether enough information exists
       ↓
5. search_policy("delayed shipment cancellation")
       ↓
6. Explain options
```

---

123. Should It Immediately Call `cancel_order`?

No.

Why?

Customer said:

> If it won't arrive tomorrow...

This is conditional.

The agent first needs verified shipping information and policy information.

---

124. If Customer Later Says

> Yes, cancel it.

The agent still must check:

```
Cancellation Eligibility
+
Permissions
+
Approval Rules
```

before executing.

---

125. Sensitive Action Flow

```
Agent Requests:
cancel_order(A123)
       ↓
Approval Required
       ↓
Human
   ┌────┴────┐
Approve    Reject
   ↓          ↓
Cancel     No Action
```

---

126. Mini Project Tool Schema Example

Conceptual:

```
{
  "name": "get_order",
  "description": "Retrieve the latest order record for a known order ID.",
  "parameters": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "description": "The exact customer order identifier."
      }
    },
    "required": [
      "order_id"
```

```
    ]
  }
}
```

This mirrors the schema-based design used in modern function calling.

---

127. Better Tool Description

Include when to use it:

```
Retrieve the latest order record for a known order ID.

Use this when the user or another tool provides a specific order ID.

Do not use this to search for orders when no order ID is available.
```

---

128. Tool Selection Exercise

Available:

```
get_customer
get_order
search_orders
get_tracking
search_policy
check_inventory
```

Match each request.

Request A

> Where is order #ABC123?

Answer:

```
get_order
```

Request B

> I don't know my order number, but my email is john@example.com.

Likely:

```
get_customer
↓
search_orders
```

Request C

> Can I return an opened product?

Answer:

```
search_policy
```

Request D

> Do you have a black XL jacket?

Answer:

```
check_inventory
```

Request E

> Thank you for your help.

Answer:

```
No tool required
```

---

129. Error Exercise

Tool:

```
get_order("A123")
```

returns:

```
{
  "success": false,
  "error": {
    "code": "SERVICE_UNAVAILABLE"
  }
}
```

What should agent do?

Possible:

```
Retry according to policy
```

If still unavailable:

```
Explain temporary problem
+
Create support ticket / escalate
```

Not:

```
Invent order information
```

---

130. Day 9 Quiz — 57 to 60 Minutes

Question 1

What is an AI tool?

A. Only a prompt
B. A capability the model can request to obtain data or perform an action
C. A database only
D. A model parameter

Answer: B

Question 2

Who should actually execute an application-defined function?

A. The model's imagination
B. The application/system

Answer: B

Question 3

What defines structured tool arguments?

A. Random text
B. Schema

Answer: B

Question 4

Which is a read tool?

```
get_order()
```

Question 5

Which is a write tool?

```
cancel_order()
```

Question 6

Why are write tools more sensitive?

Because they change external state.

Question 7

What should happen if a high-risk tool requires approval?

```
Pause
↓
Human Approval
↓
Execute / Reject
```

Question 8

What is a good tool name?

A. `thing()`
B. `get_order_by_id()`

Answer: B

Question 9

Should an agent use every available tool on every request?

Answer: No.

Question 10

What should happen when a tool fails?

A. Invent a result
B. Handle the error explicitly

Answer: B

---

131. True or False

A tool description can influence tool selection.

True

Every tool should be a write tool.

False

A function tool can use structured input parameters.

True

A tool result should ideally be predictable.

True

A failed API call means the agent should guess the answer.

False

Retrying write actions can require extra protection.

True

Tool calls should be logged in production systems.

True

An AI agent should receive every company permission.

False

RAG can be exposed to an agent as a search tool.

True

An n8n workflow can represent a higher-level business tool.

True

---

132. Day 9 Vocabulary

Tool

A capability available to an AI system.

Function Calling

A mechanism where the model requests execution of a defined function using structured arguments.

Tool Call

The model's request to use a particular tool.

Tool Output

The result returned after execution.

Parameter

An input required by a tool.

Schema

The required structure of tool arguments.

Read Tool

A tool that retrieves data.

Write Tool

A tool that changes external state.

Side Effect

An external change caused by a tool.

Approval Gate

A control that requires authorization before execution.

Timeout

A tool taking longer than an allowed limit.

Retry

Attempting a failed operation again.

Idempotency

Protection against accidentally performing the same side-effecting action multiple times.

Error Code

A predictable identifier describing a tool failure.

Tool Chaining

Using multiple tools sequentially to solve a task.

Observability

The ability to inspect what tools were called and what happened.

---

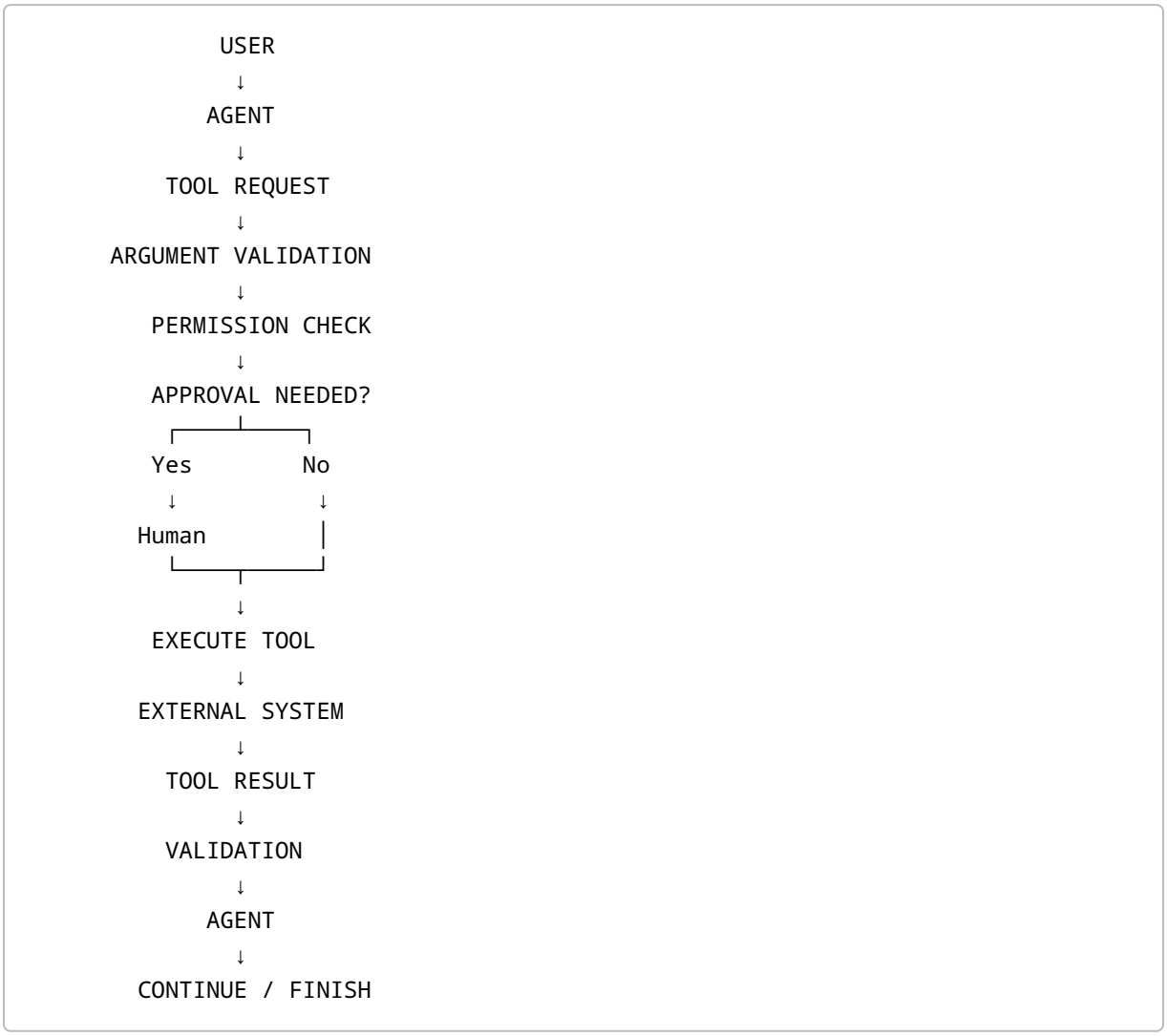
133. Five Layers of a Tool Call

Students should memorize:

```
1. TOOL DEFINITION
        ↓
2. MODEL SELECTS TOOL
        ↓
3. ARGUMENT VALIDATION
        ↓
4. APPLICATION EXECUTION
        ↓
5. TOOL RESULT
        ↓
MODEL CONTINUES
```

---

134. Professional Tool Architecture



---

135. Day 9 Golden Rules

Rule 1

Give every tool one clear responsibility.

Rule 2

Use clear names and descriptions.

Rule 3

Use structured parameters.

Rule 4

Validate arguments before execution.

Rule 5

Distinguish read and write tools.

Rule 6

Protect sensitive write actions.

Rule 7

Return structured success and error results.

Rule 8

Never turn a tool failure into a hallucinated fact.

Rule 9

Do not give agents unnecessary permissions.

Rule 10

Log important tool calls.

---

136. Day 9 Tool Design Checklist

For every tool ask:

- [ ] What is the tool name?
- [ ] Is the name unambiguous?
- [ ] What does it do?
- [ ] When should the agent use it?
- [ ] When should it NOT use it?
- [ ] What inputs are required?

- [ ] What inputs are optional?
- [ ] What data types are required?
- [ ] What enums apply?
- [ ] What does success return?
- [ ] What does failure return?
- [ ] Is it read or write?
- [ ] Does it create a side effect?
- [ ] Does it require approval?
- [ ] Can it safely be retried?
- [ ] Does it need idempotency?
- [ ] What timeout applies?
- [ ] How will it be logged?
- [ ] How will it be tested?

---

137. Homework Assignment 1 — Build a Tool Catalog

Create at least 10 tools for an ecommerce support agent.

Suggested:

```
get_customer
get_order
search_orders
get_tracking
check_inventory
search_policy
create_ticket
send_email
cancel_order
issue_refund
```

For each write:

```
Name
Description
Inputs
Output
Read / Write
Risk
Approval
```

138. Homework Assignment 2 — Design Tool Schemas

Create JSON-style parameter schemas for:

```
get_order
check_inventory
create_support_ticket
issue_refund
```

Example:

```
{
  "order_id": "string"
}
```

Then make it more precise.

---

139. Homework Assignment 3 — Design Tool Results

For each tool create:

Success

```
{
  "success": true,
  "data": {}
}
```

Failure

```
{
  "success": false,
  "error": {
    "code": "",
```

```
    "message": ""
  }
}
```

---

140. Homework Assignment 4 — Classify Tool Risk

Create:

Tool	Read/Write	Risk	Approval

Classify all 10 tools.

---

141. Homework Assignment 5 — Create Five Tool Chains

Example:

```
Customer asks order location

get_order
↓
get_tracking
↓
Answer
```

Create five business scenarios involving at least two tools.

---

142. Homework Assignment 6 — Error Handling

For each error:

```
ORDER_NOT_FOUND
TIMEOUT
UNAUTHORIZED
```

```
SERVICE_UNAVAILABLE
INVALID_ARGUMENT
```

Define:

```
Retry?
Ask User?
Escalate?
Stop?
```

---

143. Homework Assignment 7 — Build a Read-Only Agent First

Design an agent with only:

```
get_customer
get_order
get_tracking
check_inventory
search_policy
```

No write actions.

This is the safest first agent architecture.

---

144. Phase 2 Agent

After the read-only agent works, add:

```
create_ticket
```

Then later:

```
send_email
```

Then sensitive operations with approval:

```
cancel_order
issue_refund
```

---

145. Recommended Agent Development Progression

```
PHASE 1
Read Tools
   ↓
PHASE 2
Read + Structured Recommendation
   ↓
PHASE 3
Low-Risk Write Tools
   ↓
PHASE 4
Human-Approved Sensitive Tools
   ↓
PHASE 5
More Autonomous Workflows
```

---

146. Interview Questions

What is function calling?

A mechanism that allows a model to request application-defined functions with structured arguments.

What is a tool call?

A model-generated request to use a specific tool.

Who executes the function?

The application or tool runtime.

Why use JSON Schema?

To define predictable tool arguments.

What is the difference between a read and write tool?

A read tool retrieves information; a write tool changes external state.

What is idempotency?

A design property that helps prevent duplicate side effects when the same operation is repeated.

Why should sensitive tools require approval?

Because they may cause financial, security, or irreversible business consequences.

Why are tool descriptions important?

They help the model understand when and how a tool should be used.

Why return structured tool errors?

So agents and workflows can react predictably.

What is tool chaining?

Using the result of one tool to determine or supply another tool call.

---

147. Outsourcing Job Perspective

Imagine a client says:

> "I want an AI agent that connects to Shopify."

A beginner freelancer might say:

> I'll connect GPT to Shopify.

A stronger AI Automation Architect asks:

```
Which Shopify information should the agent read?

Which Shopify actions may it perform?

What tools are required?

Which actions have side effects?

Which actions need approval?

What parameters does each tool require?

What should happen when Shopify is unavailable?

What should happen if an order cannot be found?

How do we prevent duplicate actions?

How will tool calls be logged?
```

That is the difference between:

```
AI Demo
```

and:

```
Production AI Architecture
```

---

148. Day 9 Final Mental Model

Students should leave with this architecture:

```
                USER GOAL
                    ↓
                 AGENT
                    ↓
             CHOOSE TOOL?
              ┌─────┴─────┐
            No            Yes
             ↓             ↓
```

```
          Respond      Build Arguments
                              ↓
                          Validate
                              ↓
                     Permission Check
                              ↓
                     Approval Needed?
                       ┌──────┴──────┐
                     Yes             No
                       ↓             ↓
                    Human            │
                       └──────┬──────┘
                              ↓
                        Execute Tool
                              ↓
                       External System
                              ↓
                         Tool Result
                              ↓
                         Validate
                              ↓
                           Agent
                              ↓
                    Continue / Finish
```

---

149. Day 9 Final Takeaway

Students should remember:

> **An AI tool is the bridge between model reasoning and real software capability.**

But that bridge needs control.

The professional formula is:

```
AI Agent
+
Well-Designed Tools
+
Structured Schemas
+
Validation
+
```

```
Permissions
+
Error Handling
+
Human Approval
+
Logging
=
Reliable Agentic System
```

The biggest mistake is:

```
Give AI powerful tools
+
No controls
```

The better approach is:

```
Give AI narrow tools
+
Clear responsibilities
+
Minimum permissions
+
Deterministic safeguards
```

Day 9 Learning Outcome Checklist

Students should now be able to:

- [ ] Define an AI tool
- [ ] Explain function calling
- [ ] Explain the full tool-call lifecycle
- [ ] Design a clear tool name
- [ ] Write a useful tool description
- [ ] Define required parameters
- [ ] Understand JSON Schema for tool inputs
- [ ] Use enums
- [ ] Distinguish read and write tools
- [ ] Explain side effects
- [ ] Identify sensitive tools
- [ ] Define approval rules

- [ ] Design structured success output
- [ ] Design structured error output
- [ ] Explain retries
- [ ] Explain timeouts
- [ ] Explain idempotency
- [ ] Explain tool chaining
- [ ] Explain API-backed tools
- [ ] Explain RAG as a tool
- [ ] Explain database tools
- [ ] Explain n8n workflow tools
- [ ] Design a support-agent tool catalog
- [ ] Explain why narrow tools are safer
- [ ] Explain why tools need logging and testing

If students can design the **Customer Support Agent Tool Catalog** and explain when each tool should or should not be called, **Day 9 has succeeded.**

Preview — Day 10

Day 10 — Function Calling in Practice: Building an Agent That Uses Tools

Day 10 will move from:

```
Designing Tools
```

to:

```
Actually Connecting Tools to GPT
```

Students will learn:

- Tool definition in code
- OpenAI API tool calling
- Responses API concept
- Tool-call objects
- `call_id`
- Parsing arguments
- Executing Python functions
- Returning `function_call_output`
- Multi-step tool calls

- Agent loops
- Tool-choice behavior
- Handling malformed inputs
- Error handling
- Read-only support agent
- Testing function calls
- Logging
- Basic security
- Comparing raw function calling with an Agent SDK
- When to use the OpenAI Agents SDK
- When to use n8n tools
- How tool calls connect to real APIs

The Day 10 practical architecture will be:

```
User
 ↓
GPT / Agent
 ↓
Tool Call
 ↓
Python Function
 ↓
Mock Order Database
 ↓
Tool Result
 ↓
GPT
 ↓
Final Customer Response
```

The main project will be:

Build a Working Python Order-Support Agent

with tools such as:

```
get_order()
get_tracking()
search_policy()
```

Day 10 will therefore be the first point where students move from **agent architecture diagrams** to a **working tool-calling implementation**.